JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Class ThreadLocal<T>

java.lang.Object
    java.lang.ThreadLocal<T>

Direct Known Subclasses:

InheritableThreadLocal

```
public class ThreadLocal<T>
extends Object
```

This class provides thread-local variables. These variables differ from their normal counterparts in that each thread that accesses one (via its `get` or `set` method) has its own, independently initialized copy of the variable. `ThreadLocal` instances are typically private static fields in classes that wish to associate state with a thread (e.g., a user ID or Transaction ID).

For example, the class below generates unique identifiers local to each thread. A thread's id is assigned the first time it invokes `ThreadId.get()` and remains unchanged on subsequent calls.

```
import java.util.concurrent.atomic.AtomicInteger;

public class ThreadId {
    // Atomic integer containing the next thread ID to be assigned
    private static final AtomicInteger nextId = new AtomicInteger(0);

    // Thread local variable containing each thread's ID
    private static final ThreadLocal<Integer> threadId =
        new ThreadLocal<Integer>() {
            @Override protected Integer initialValue() {
                return nextId.getAndIncrement();
        }
    };
```

```
    // Returns the current thread's unique ID, assigning it if necessary
    public static int get() {
        return threadId.get();
    }
}
```

Each thread holds an implicit reference to its copy of a thread-local variable as long as the thread is alive and the `ThreadLocal` instance is accessible; after a thread goes away, all of its copies of thread-local instances are subject to garbage collection (unless other references to these copies exist).

Since:

1.2

Constructor Summary

Constructors

Constructor	Description
`ThreadLocal()`	Creates a thread local variable.

Method Summary

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
`T`	`get()`	Returns the value in the current thread's copy of this thread-local variable.
`protected T`	`initialValue()`	Returns the current thread's "initial value" for this thread-local variable.
`void`	`remove()`	Removes the current thread's value for this

		thread-local variable.
void	**set**(**T** value)	Sets the current thread's copy of this thread-local variable to the specified value.
static <S> **ThreadLocal**<S>	**withInitial**(**Supplier**<? extends S> supplier)	Creates a thread local variable.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

ThreadLocal

```
public ThreadLocal()
```

Creates a thread local variable.

See Also:

withInitial(java.util.function.Supplier)

Method Detail

initialValue

```
protected T initialValue()
```

Returns the current thread's "initial value" for this thread-local variable. This method will be invoked the first time a thread accesses the variable with the `get()` method, unless the thread previously invoked the `set(T)` method, in which case the `initialValue` method will not be invoked for the thread. Normally, this method is invoked at most once per thread, but it may be invoked again in case of subsequent invocations of `remove()` followed by `get()`.

This implementation simply returns `null`; if the programmer desires thread-local variables to have an initial value other than `null`, `ThreadLocal` must be

subclassed, and this method overridden. Typically, an anonymous inner class will be used.

Returns:

the initial value for this thread-local

withInitial

```
public static <S> ThreadLocal<S> withInitial(Supplier<? extends S> supplier)
```

Creates a thread local variable. The initial value of the variable is determined by invoking the `get` method on the `Supplier`.

Type Parameters:

`S` - the type of the thread local's value

Parameters:

`supplier` - the supplier to be used to determine the initial value

Returns:

a new thread local variable

Throws:

`NullPointerException` - if the specified supplier is null

Since:

1.8

get

```
public T get()
```

Returns the value in the current thread's copy of this thread-local variable. If the variable has no value for the current thread, it is first initialized to the value returned by an invocation of the `initialValue()` method.

Returns:

the current thread's value of this thread-local

set

```
public void set(T value)
```

Sets the current thread's copy of this thread-local variable to the specified value. Most subclasses will have no need to override this method, relying solely on the `initialValue()` method to set the values of thread-locals.

Parameters:

value - the value to be stored in the current thread's copy of this thread-local.

remove

```
public void remove()
```

Removes the current thread's value for this thread-local variable. If this thread-local variable is subsequently read by the current thread, its value will be reinitialized by invoking its `initialValue()` method, unless its value is set by the current thread in the interim. This may result in multiple invocations of the `initialValue` method in the current thread.

Since:

1.5

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.